

Inference to Tensors

Implemented Batch 2 of the curriculum and fixed the Batch 1 visual/PDF layout issues with HTML legends, numbered callouts, and print-safe captions.

What Changed

- Implemented full cards for Inference, Prompt vs Response, Tokenization, Token IDs, Embeddings, Vectors, and Tensors.
- Bumped the visible app version to `v0.9.0`.
- Moved lessons 8-14 into the complete lesson schema: definition, core explanation, where it happens, durable/temp, prompt/response, metaphor, Brain Bridge, misconception, visual aid, and checkpoint.
- Improved Batch 1 visuals for Rationalists vs Empiricists, Training, Overfitting, and Alignment.
- Updated the glossary for tokenizer, tokenization, embedding table, distributed representation, activation, hidden state, and weight tensor.
- Updated the default lesson-card PDF path to `docs/review/prompt-life-lesson-cards-v0-9.pdf`.
- Generated the v0.9 lesson-card PDF at `docs/content-inventory/prompt-life-lesson-cards-v0-9-batch-2.pdf`.

How To Run Locally

From this repo:

```
cd ~/Desktop/promptlife
npm install
npm run dev
```

Open `http://localhost:5173/`. For same-Wi-Fi phone testing, run `npm run dev -- --host 0.0.0.0` and open the Network URL Vite prints.

Verification

- `npm install` : passed.
- `npm run build` : passed.
- `npm run export:lesson-cards` : passed.
- Review route: 28 lesson cards, Batch 2 present.
- No new games, Three.js, or heavy 3D library added.
- `npm run typecheck` : passed.
- `npm run export:lesson-pdf` : passed.
- Mobile overflow audit: passed at 320px, 390px, and 430px.
- PDF sample page: no clipped visual, vertical caption, or unreadable label found.

Files Changed

`README.md` , `package.json` , `scripts/export-lesson-pdf.mjs` , `src/components/VisualAids.tsx` , `src/data/content.ts` , `src/main.tsx` , `src/styles/global.css` , `docs/REVIEW_NOTES.md` , Batch 2 curriculum docs, v0.9 lesson-card PDFs, and v0.9 screenshot artifacts.

Next Three v0.10 Improvements

1. Implement Batch 3 for Attention, MLP, Layers, and Hidden States as a connected transformer-workday section.
2. Add source citations or a bibliography layer for Batch 1 and Batch 2 before broader publication.
3. Add an automated screenshot/overflow script to the repo so future visual-aid changes can be checked repeatably.

Screenshots

MAIN PATH

Today in Prompt Life

Follow one prompt from before training through inference, generation, and model literacy.

1

Before Morning

2

Morning Commute

3

Workday

4

Decision Room

5

The Day Repeats

6

Wider AI Literacy

GUIDED COMPARISONS

Run and compare

Use these when the map feels abstract: one guides a prompt through the model, the other compares how AI systems learn.

Prompt Run

How AI Learns

1

Before Morning

What an LLM is, where it came from, and how training, pretraining, overfitting, fine-tuning, and alignment shape it before ordinary use.

The prompt has not arrived yet. The model is being shaped before ordinary

USE



Home



Journey



Play



Glossary

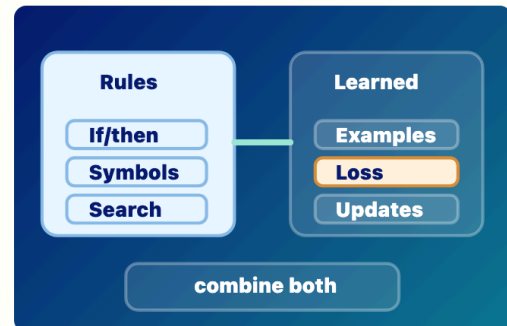


Badge

Journey top

VISUAL AID

What to picture



Rules and Learned Patterns

Symbolic/rationalist systems use explicit rules; deep-learning/empiricist systems learn useful patterns from examples.

- 1 Rules: if/then logic and symbols.
- 2 Examples: data, loss, and weight updates.
- 3 Modern AI products can combine learned models with rules and tools.

CORE IDEA

Core idea

Early AI often followed the rationalist dream: intelligence as explicit rules, logic, symbols, and search. Deep learning follows a more empiricist path: expose a model to many examples, measure error, and let the system learn internal representations that work.



Home



Journey



Play



Glossary



Badge

Batch 1 fixed: Rationalists vs Empiricists

VISUAL AID

What to picture



Training Loop

Predict, compare, loss, update weights, repeat. Training changes weights.

- 1 Predict and compare against a target.
- 2 Loss measures error.
- 3 Weight updates make training durable.

CORE IDEA

Core idea

During training, the model sees examples, predicts something, measures how wrong it was, and updates its weights to do slightly better next time. This happens over many examples and many updates. Training is where durable model capability is created.

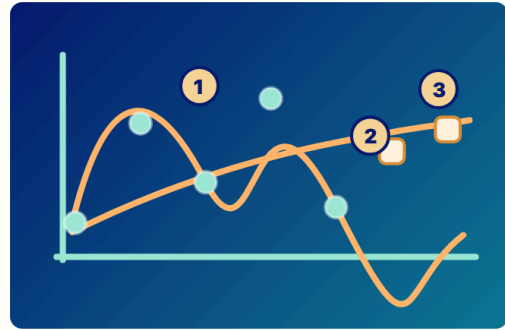
MODEL LIFECYCLE

Home Journey Play Glossary Badge

Batch 1 fixed: Training Loop

VISUAL AID

What to picture



Overfitting vs Generalization

Memorizing examples is not the same as learning transferable patterns.

- 1 Overfit curve traces old dots too tightly.
- 2 Generalizing curve is smoother and reaches new examples.
- 3 Held-out examples test whether learning transfers.

CORE IDEA

Core idea

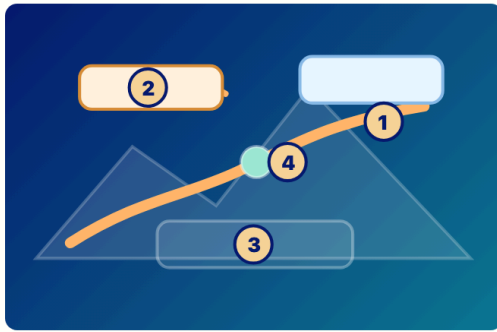
A model can fit old examples very well but still fail on new cases. That is overfitting. Good learning captures patterns that transfer. For LLMs, generalization is why a model can answer prompts it has never seen exactly before. Overfitting is why evaluation

Home Journey Play Glossary Badge

Batch 1 fixed: Overfitting

VISUAL AID

What to picture



Alignment Landscape

Alignment shapes behavior with preferred paths, guardrails, feedback, and policies; it does not create moral agency.

- 1 Preferred path: behavior the system encourages.
- 2 Warning zone: outputs to avoid or handle carefully.
- 3 Policy check: runtime or review constraint.
- 4 Feedback: behavior signal, not conscience.

CORE IDEA

Core idea

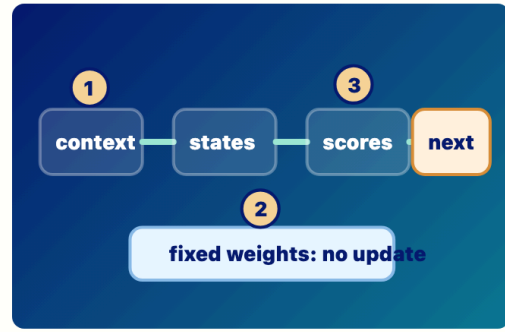
A model trained only to predict likely text may produce outputs that are fluent but unhelpful, unsafe, biased, or misleading. Alignment uses methods such as instruction

Home Journey Play Glossary Badge

Batch 1 fixed: Alignment

VISUAL AID

What to picture



Forward Pass

Inference creates temporary states while durable weights stay fixed.

- 1 Current context enters the model.
- 2 Fixed weights are used, not updated.
- 3 Temporary hidden states lead to next-token scores.

CORE IDEA

Core idea

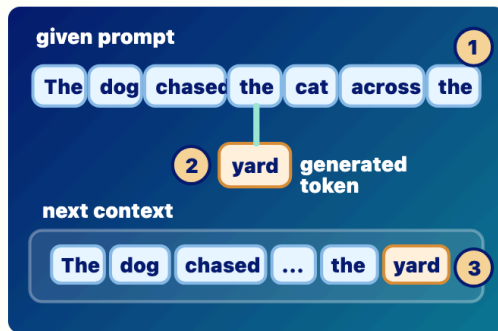
During ordinary use, the current context enters the model, embeddings and hidden states are computed, the final hidden state produces next-token scores, and decoding chooses one response token. Then the token is appended and the process repeats. The learned weights are used, but they normally

Home Journey Play Glossary Badge

Batch 2: Inference

VISUAL AID

What to picture



Prompt vs Response

Prompt tokens are given; response tokens are generated and appended.

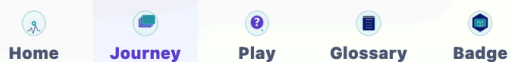
- 1 Prompt tokens are supplied context.
- 2 The generated token 'yard' is appended.
- 3 The next run sees prompt plus response so far.

CORE IDEA

Core idea

The model does not process the prompt once and then write a whole answer. It repeatedly runs on the current context. At first, the context contains the prompt and any prior conversation or retrieved material. After the model chooses one response token, that token is appended. The next

forward pass sees the prompt plus the



Batch 2: Prompt vs Response

VISUAL AID

What to picture



Text to Tokens

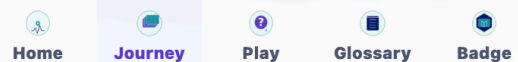
Text is split into model-readable chunks before embedding lookup.

- 1 Tokens can be words, word pieces, punctuation, or spaces.
- 2 This app uses simplified tokens; real tokenizers may split differently.

CORE IDEA

Core idea

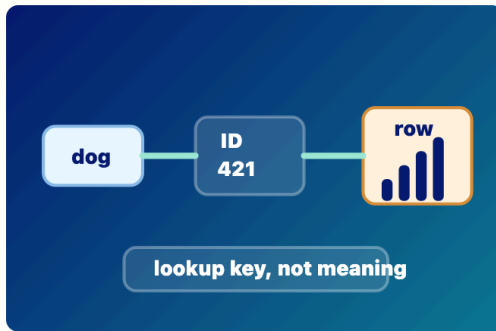
LLMs do not carry raw English sentences through their layers. Text is first split into tokens: pieces that might be whole words, word parts, punctuation, or spaces. Prompt text is tokenized before the model processes it. Generated response tokens are also token IDs before they are turned back into visible



Batch 2: Tokenization

VISUAL AID

What to picture



Token IDs

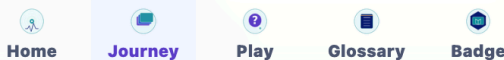
Each token gets a lookup number that points to an embedding row.

- 1 The token is not carried forward as raw English.
- 2 The ID is a lookup key, not the meaning.
- 3 The ID points to an embedding row.

CORE IDEA

Core idea

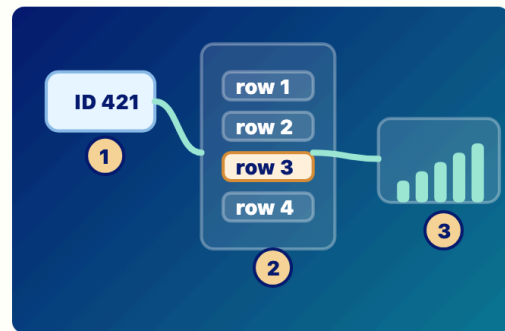
After tokenization, each token is represented by an integer ID. The ID is not the meaning. It is a lookup key. The embedding table uses that ID to retrieve the token's learned starting vector. During generation, the model also chooses a next token ID before it is displayed as text.



Batch 2: Token IDs

VISUAL AID

What to picture



Embedding Lookup

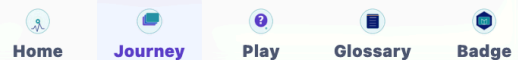
A token ID retrieves a learned starting vector.

- 1 The embedding table was learned during training.
- 2 Inference retrieves one row temporarily for the current context.
- 3 Later layers reshape embeddings into hidden states.

CORE IDEA

Core idea

The embedding table is learned during training. At inference time, each token ID in the current context retrieves one row from that table. That row is the token's starting vector before context changes it. Later layers turn embeddings into hidden states



Batch 2: Embeddings

VISUAL AID

What to picture



Feature Vector

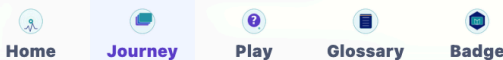
A vector is a list of numerical features, not a sentence.

- 1 Slider labels are simplified teaching examples.
- 2 Real features are distributed across many dimensions.

CORE IDEA

Core idea

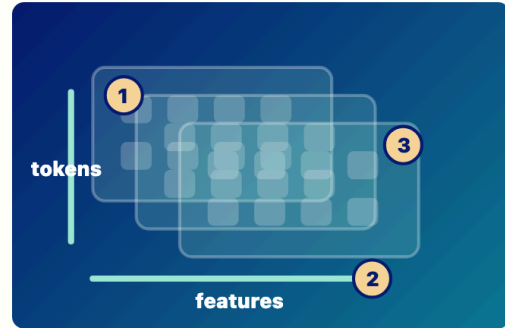
In an LLM, many things are represented as vectors. An embedding is a starting vector for a token ID. A hidden state is a later vector shaped by context. The numbers are not usually neat labeled features like dogness or grammar. Features are distributed across many dimensions, which is why vectors can represent fuzzy relationships.



Batch 2: Vectors

VISUAL AID

What to picture



Tensor Block

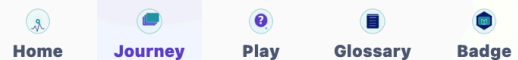
Tensors organize many token vectors for layer-by-layer processing.

- 1 Token positions run down one axis.
- 2 Feature dimensions run across another axis.
- 3 Batch x tokens x features is an advanced shape note.

CORE IDEA

Core idea

Once tokens have embeddings, the model needs to carry many token positions and many features together. A simple way to picture this is a table: token positions down one side, feature numbers across the other. With batches, heads, and layers, the shapes get richer. But the core idea is simple:



Batch 2: Tensors

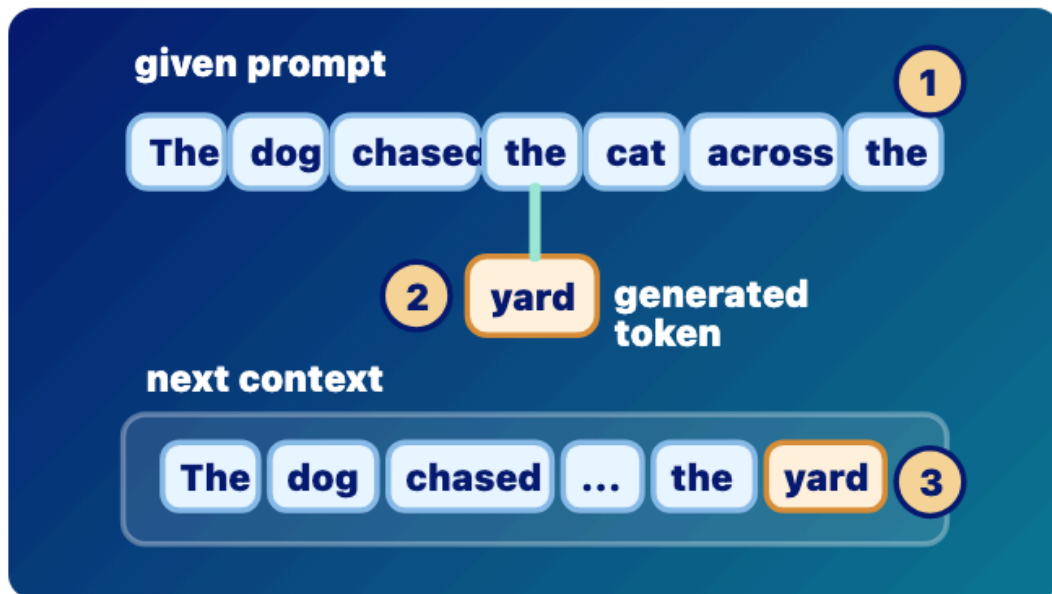
Prompt vs Response

Given context versus generated tokens

Stage: response-generation

Priority: low

Rubric avg: 4.4/5



Prompt vs Response

Prompt tokens are given; response tokens are generated and appended.

- 1 Prompt tokens are supplied context.
- 2 The generated token yard is appended.
- 3 The next run sees prompt plus response so far.

DEFINITION

Prompt tokens are given to the model:

Review route sample

Prompt Life

10 / 29

92%

7

8

9

10

11

9 / 28

Prompt vs Response

Given context versus generated tokens

Stage: response-generation Priority: low Rubric avg: 4.4/5

DEFINITION
Prompt tokens are given to the model; response tokens are generated by the model one at a time and appended to the context.

DURABLE VS TEMPORARY
Prompt and response-so-far tokens are temporary context. They can influence the current run, but they do not normally update weights.

RELATIONSHIP
After we distinguish prompt from response, we can see how both are represented as tokens.

BRAIN LIMIT
A person may plan, intend, and revise. The model repeatedly samples next tokens from probabilities.

CURRENT EXERCISE
None rendered in journey; no direct Play mapping.

REWRITE NOTES
Risk: The model generates the whole response at once. **Missing:** The model does not process the prompt once and then write a whole answer. It repeatedly runs on the current context. At first, the context contains the prompt and any prior conversation or retrieved material. After the model chooses one response token, that token is appended. The next forward pass sees the prompt plus the response so far.

CORE EXPLANATION
The model does not process the prompt once and then write a whole answer. It repeatedly runs on the current context. At first, the context contains the prompt and any prior conversation or retrieved material. After the model chooses one response token, that token is appended. The next forward pass sees the prompt plus the response so far.

PROMPT VS RESPONSE
This is the distinction itself: given tokens are prompt/context; generated tokens become the response.

METAPHOR
Given cards on a table, then new cards added one at a time.

MISCONCEPTION
The model generates the whole response at once.

CHECKPOINT
What happens after one response token is generated? Correct: It is appended to the context. Incorrect: The whole answer appears at once. The model permanently learns it; Softmax disappears.

ILLUSTRATION NEEDED
Current visual aid: prompt-response. Given prompt cards and newly generated response cards entering the next context together.

WHERE IT HAPPENS
During inference and response generation.

WHY IT MATTERS
It prevents the myth that the model writes a full answer all at once.

BRAIN METAPHOR
Like hearing a sentence and continuing it word by word.

VISUAL AID
prompt-response

FEEDBACK
The response grows by next token, append, repeat. Choice notes: The whole answer appears at once. Not quite. A generated token becomes part of the next context, but it is not a durable weight update. The model permanently learns it; Not quite. A generated token becomes part of the next context, but it is not a durable weight update. Softmax disappears. Not quite. A generated token becomes part of the next context, but it is not a durable weight update.

Prompt vs Response
Prompt tokens are given; response tokens are generated and appended.
1 Prompt tokens are supplied context.
2 The generated token yard is appended.
3 The next run sees prompt plus response so far.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 5/5	Prompt/response 5/5	Durable/temp 4/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 4/5

prompt

response

prompt-tokens

response-tokens

generated-token

input-context

completion

10 / 28

Tokenization

Text becomes model-readable chunks

MORNING COMMUTE

PDF sample page